

电子科技大学

实验报告

一、实验项目名称： 实验3-1-PE文件入口点修改、实验3-2-病毒尾区段寄生实验
(大小不变)

二、实验目的：

本实验围绕 Windows PE 可执行文件的结构分析与入口控制机制展开，目标包括以下几个方面：

1. 掌握 PE 文件中 DOS 头、NT 头、可选映像头和节表之间的基本关系，理解操作系统加载可执行文件时并不是简单顺序执行文件内容，而是依赖 PE 结构中的关键字段完成装载。
2. 理解 AddressOfEntryPoint 字段的作用，掌握程序入口 RVA 与运行时虚拟地址、文件偏移之间的区别，并能够通过节表字段完成 RVA 到文件偏移的换算。
3. 学会使用 PEView、ImHex 和 x32dbg 对 PE 文件进行静态分析、字节修改和动态调试验证，形成“结构定位-字段修改-运行验证”的实验闭环。
4. 理解尾区段寄生的基本原理，掌握 VirtualSize、Size of Raw Data、Pointer to Raw Data 等字段在节加载过程中的作用，说明为什么简单把代码追加到文件尾部不能保证其被加载和执行。
5. 通过入口点修改和尾区段寄生实验，认识 PE 结构字段对程序执行流程的控制作用，为后续恶意代码分析、程序逆向和软件安全实验奠定基础。

三、实验原理：

PE 文件是 Windows 系统下可执行程序的主要文件格式。一个 PE 文件并不是单纯由连续机器码组成，而是由 DOS 头、NT 头、可选映像头、节表以及各个节共同构成。程序装载时，操作系统的 PE 加载器会读取这些结构，根据其中记录的地址、大小和权限信息，将文件中的不同部分映射到进程虚拟地址空间中。

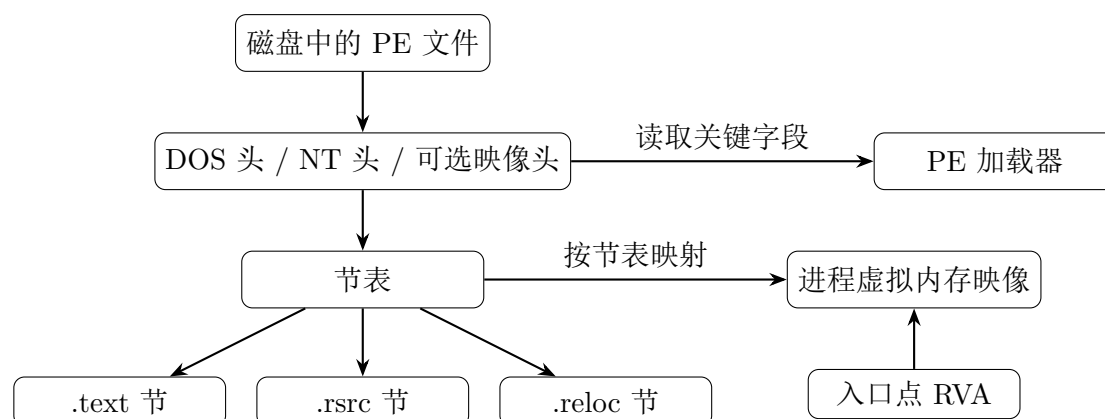


图 1 PE 文件结构与加载过程示意图

如图 1 所示，PE 文件中的可选映像头和节表是连接“磁盘文件”和“内存映像”的关键结构。可选映像头中的 `AddressOfEntryPoint` 字段保存程序入口点的 RVA，即程序装载后第一条执行指令相对于模块基址的偏移。实际运行时，入口虚拟地址等于模块加载基址加上入口 RVA。由于程序不一定总是加载到首选 `ImageBase`，因此调试器中看到的入口虚拟地址可能会变化，但其对应的 RVA 仍由 `AddressOfEntryPoint` 控制。

另一方面，十六进制编辑器修改的是磁盘文件中的字节，因此实验中不能直接把 RVA 当作文件偏移使用。需要先判断目标 RVA 落在哪一个节中，再根据该节的 RVA 和 `Pointer to Raw Data` 进行换算。其基本关系为：

$$\text{文件偏移} = \text{目标 RVA} - \text{节起始 RVA} + \text{节文件偏移}$$

本实验 3-1 正是利用这一关系，将入口点 RVA `0x1404` 换算为文件偏移 `0x0804`，并进一步修改 `AddressOfEntryPoint` 字段，使程序从新的入口位置开始执行。

实验 3-2 的核心在于 PE 文件尾区段寄生。直接把字节追加到 PE 文件末尾，并不一定会被 PE 加载器装入内存。原因是加载器并不是按文件长度简单读取全部内容，而是依据节表中 `VirtualSize`、`Size of Raw Data`、RVA 和 `Pointer to Raw Data` 等字段确定每个节在文件中和内存中的有效范围。`VirtualSize` 描述节加载到内存后的实际有效大小，`Size of Raw Data` 描述节在文件中按文件对齐后的大小。若最后一个节的 `VirtualSize` 小于 `Size of Raw Data`，则说明该节在文件对齐区域中存在空洞，可以在不增加文件长度的情况下写入少量代码。

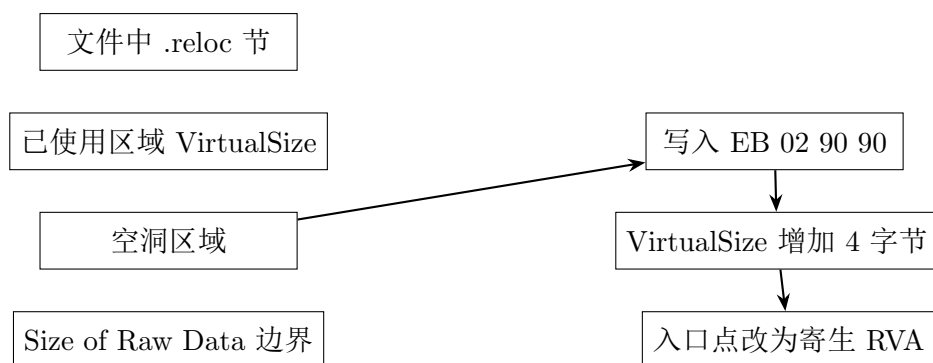


图 2 尾区段空洞寄生的字段修改逻辑

如图 2 所示，尾区段寄生并不是单纯写入机器码，而是要同时完成三个动作：第一，把寄生代码写入最后一个节的空洞区域；第二，修改该节的 VirtualSize，使新增字节被视为节的有效内容；第三，修改 AddressOfEntryPoint，使程序启动后跳转到寄生代码的 RVA。本实验采用文件大小不变的方式，只使用最后一个节已有的空洞，因此不需要新增节，也不需要修改 Size of Raw Data 或 Size of Image。

四、实验环境（设备、元器件）：

Windows 11 操作系统；PEView；ImHex；x32dbg；实验样本文件 Control.exe 及其副本文件。

五、实验步骤：

1.1 实验一：PE文件入口点修改

1.1.1 原始入口点定位

PE 文件的程序入口点由可选映像头中的 AddressOfEntryPoint 字段给出。该字段保存的是入口点的 RVA，而不是文件中的直接偏移地址。因此，实验首先使用 PEView 打开原始样本 Control.exe，在 PE 结构树中展开 NT 头和可选映像头，定位入口点字段。

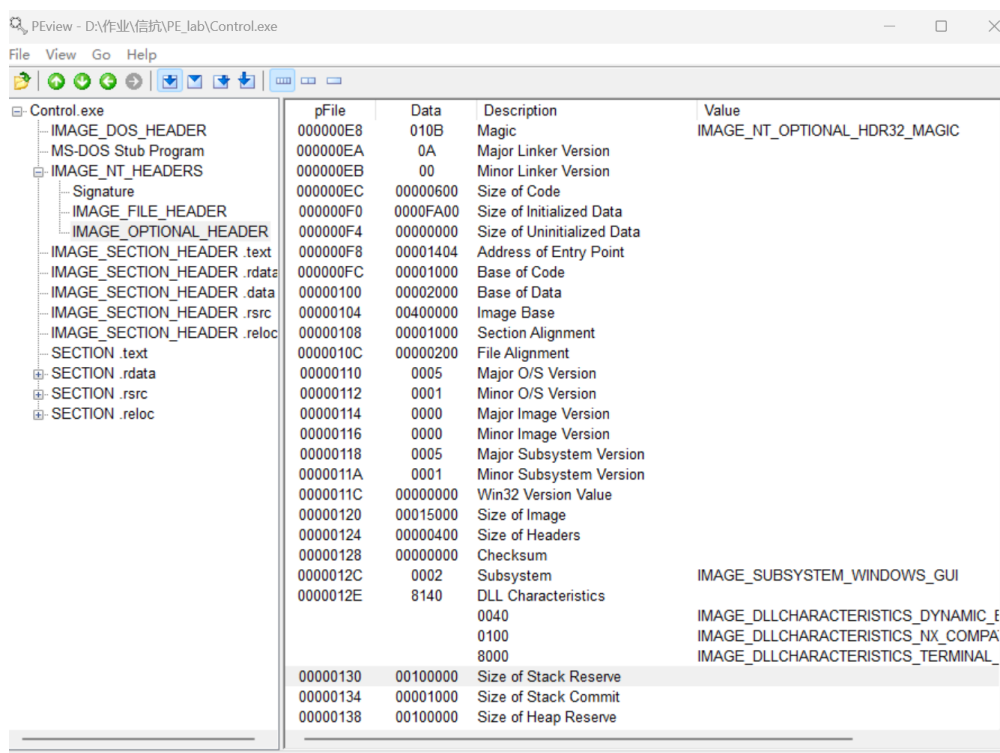


图 3 使用 PEView 查看 Control.exe 的原始入口点字段

如图 3 所示，原始样本的 Address of Entry Point 字段值为 00001404，说明程序原始入口点 RVA 为 0x1404。同时，图中可见 Image Base 为 0x00400000，File Alignment 为 0x0200，这些字段为后续地址换算和调试验证提供了依据。

1.1.2 入口点 RVA 到文件偏移的转换

十六进制编辑器按照文件偏移定位字节，而 AddressOfEntryPoint 给出的是 RVA。因此，修改前需要先结合入口点所在节的节表信息，将入口 RVA 转换为文件偏移。使用 PEView 查看 .text 节表后，可以得到 .text 节的起始 RVA、文件偏移和节大小。

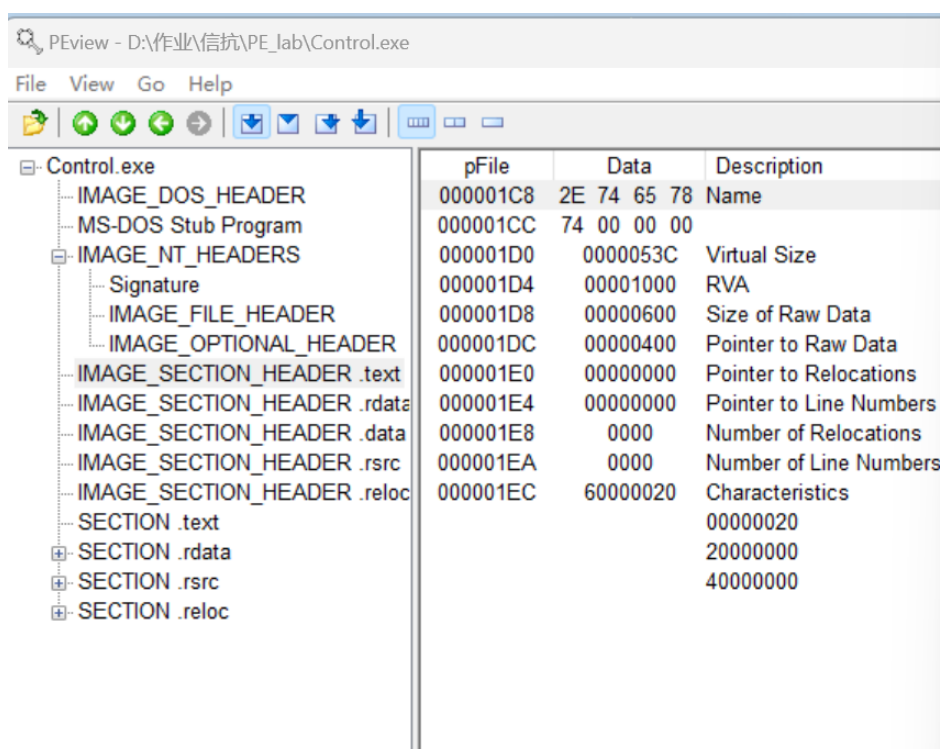


图 4 使用 PEView 查看 .text 节表字段

如图 4 所示，.text 节的 RVA 为 0x1000，Pointer to Raw Data 为 0x0400，Size of Raw Data 为 0x0600。原始入口点 0x1404 位于 .text 节范围内，因此入口点对应的文件偏移计算如下：

$$0x1404 - 0x1000 + 0x0400 = 0x0804$$

随后使用 ImHex 打开 Control.exe，跳转到文件偏移 0x0804，验证该位置是否为入口点对应的机器码。

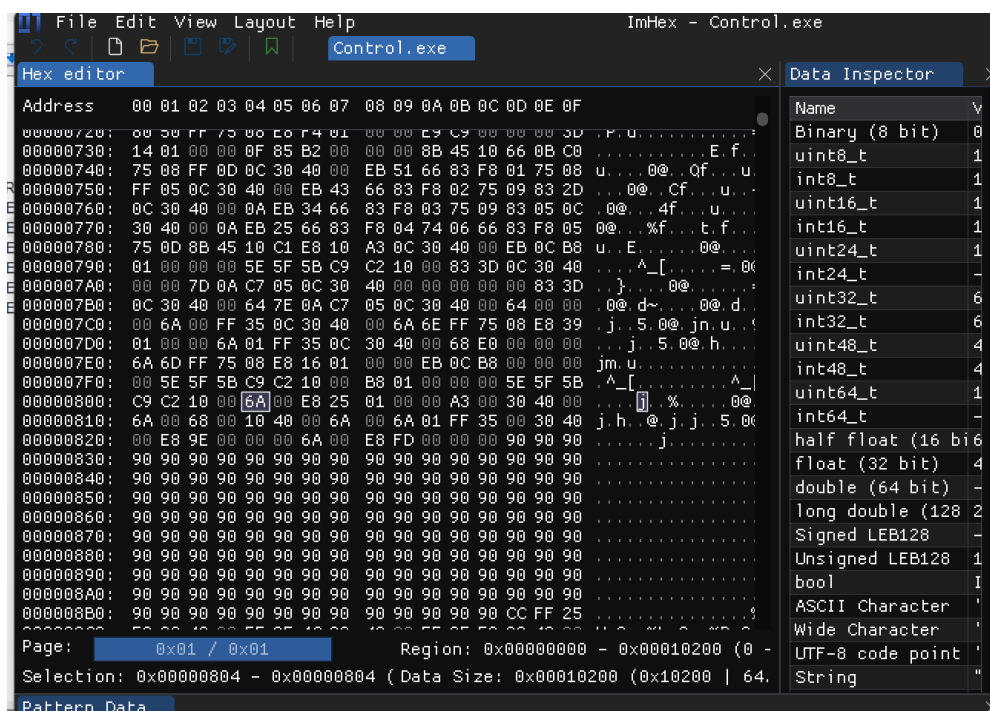


图 5 使用 ImHex 定位入口点对应的文件偏移 0x0804

如图 5 所示，ImHex 底部状态栏显示当前选中位置为 0x00000804，该位置处首字节为 6A。该结果与后续调试器中入口点处显示的机器码一致，说明入口点 RVA 到文件偏移的换算结果正确。

1.1.3 入口点字段修改

完成原始入口点定位后，对 Control.exe 的副本文件进行修改。根据前文结果，原始入口点 RVA 为 0x1404。结合调试器中入口附近的反汇编结果，本实验将入口点向后移动到 0x1419，使程序从新的指令位置开始执行。

需要注意的是，本实验修改的不是 0x0804 处的机器码，而是 PE 可选映像头中的 AddressOfEntryPoint 字段。由前文 PEView 结果可知，该字段本身在文件中的偏移为 0xF8。由于 PE 文件中的多字节整数采用小端序存储，原字段值 0x00001404 在十六进制编辑器中表现为 04 14 00 00；将入口点修改为 0x00001419 后，对应字节应修改为 19 14 00 00。

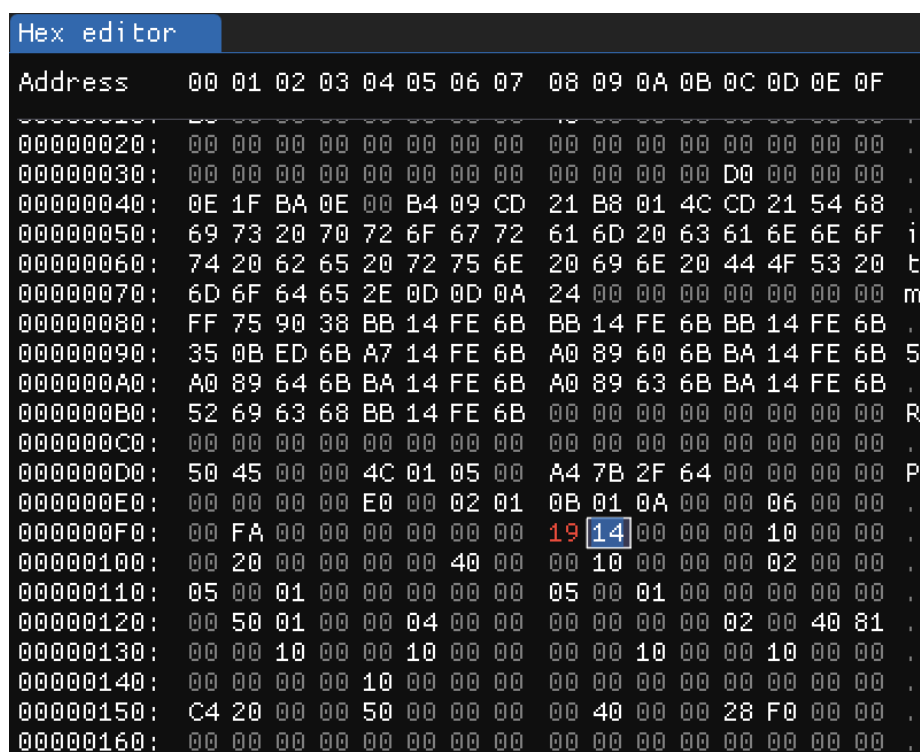
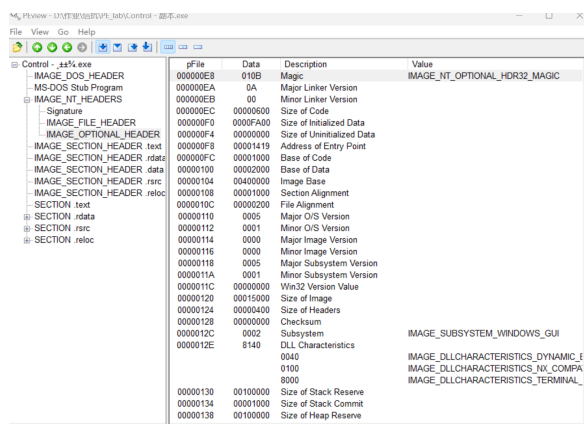


图 6 使用 ImHex 将入口点字段修改为 0x1419

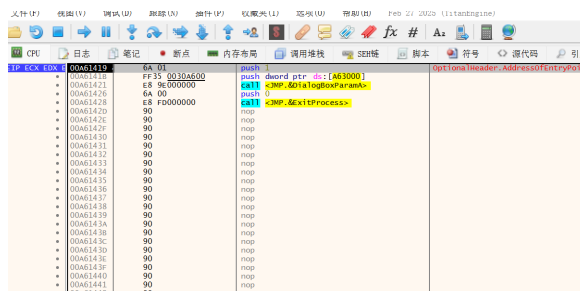
如图 6 所示，在文件偏移 0xF8 附近可以看到入口点字段已经被修改为 19 14 00 00。按小端序解释，该值对应 0x00001419，说明入口点字段已经从原始的 0x1404 修改为 0x1419。

1.1.4 修改结果验证

为确认修改是否真正写入 PE 结构，首先重新使用 PEView 打开修改后的副本文件，再次查看可选映像头中的 Address of Entry Point 字段；随后使用 x32dbg 打开修改后的程序，观察程序实际停下的位置和入口处反汇编指令。二者共同构成对实验结果的验证。



(a) PEView 验证入口点字段已修改为 0x1419



(b) x32dbg 验证修改后的程序入口点为 0x1419

图 7 入口点字段修改后的静态与动态验证结果

如图 7 所示，左图中修改后的文件里 Address of Entry Point 字段值已经变为 00001419，说明 PE 头中的入口点 RVA 已成功修改；右图中程序加载后调试器停在 00A61419 处，当前位置对应入口点 RVA 0x1419。该位置的机器码为 6A 01，反汇编结果为 push 1，与修改前入口点处的指令不同。由此可以确认，修改 AddressOfEntryPoint 字段后，程序启动时的第一条执行指令已经发生改变。

1.2 实验二：PE文件尾区段寄生实验

1.2.1 最后一个节空洞判断

本实验在 PE 文件最后一个节的空洞区域中写入少量机器码，并通过修改相关 PE 结构字段，使新增字节位于 PE 加载器认可的节范围内。若最后一个节中存在空洞，则可以在不增加文件长度的情况下写入寄生代码；此时只需要修改该节的 VirtualSize 字段，而 Size of Raw Data 保持不变。

使用 PEView 打开实验样本 Control - 副本 (2).exe，并查看最后一个节 .reloc 的节表信息。

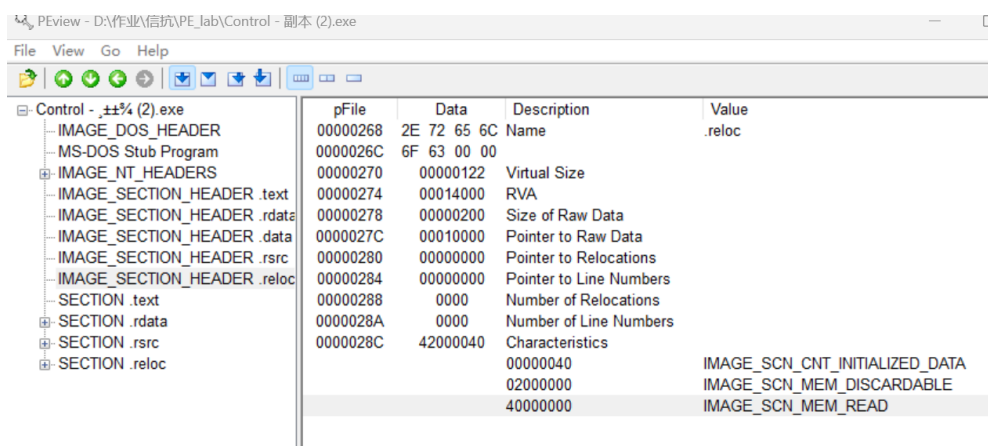


图 8 PEView 查看最后一个节 .reloc 的节表字段

如图 8 所示，.reloc 节的 Virtual Size 为 0x0122，RVA 为 0x14000，Size of Raw Data 为 0x0200，Pointer to Raw Data 为 0x10000。由于 0x0122 小于 0x0200，说明 .reloc 节在文件中存在未使用的空洞区域。空洞大小计算如下：

$$0x0200 - 0x0122 = 0x00DE$$

即 .reloc 节中仍有 0xDE 字节空间可用。本实验只需要写入 4 字节机器码，因此该空洞空间足够完成文件大小不变的尾区段寄生实验。

1.2.2 寄生位置计算

根据 .reloc 节表字段，寄生代码应写入该节已有有效数据之后。因此，寄生代码在文件中的偏移位置为：

$$0x10000 + 0x0122 = 0x10122$$

对应的 RVA 为：

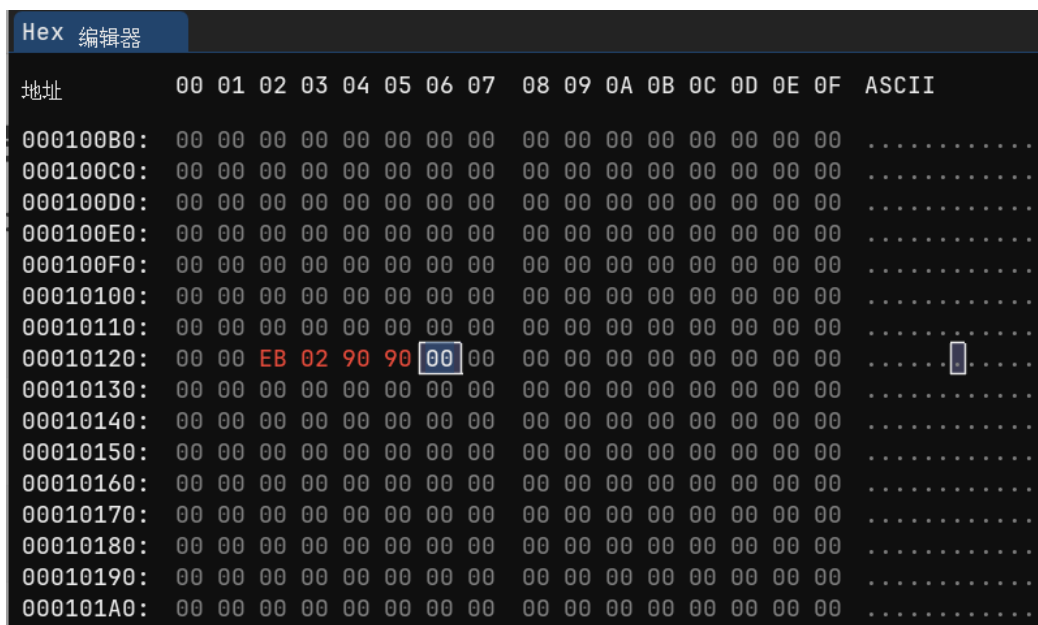
$$0x14000 + 0x0122 = 0x14122$$

由此可知，文件偏移 0x10122 是本次写入寄生机器码的位置，而 RVA 0x14122 则是后续需要写入入口点字段的目標地址。

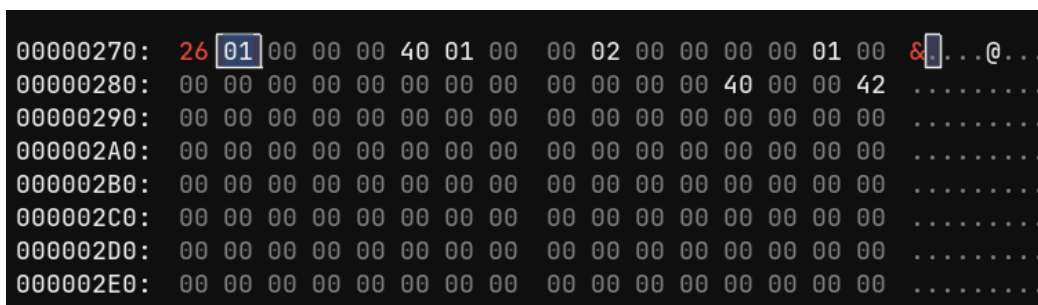
1.2.3 寄生代码写入与相关字段修改

完成寄生位置计算后，使用 ImHex 对实验样本进行三处修改。第一，在文件偏移 0x10122 处写入 4 字节机器码 EB 02 90 90。其中，EB 02 表示短跳转指令，90 90 为两条空操作指令。第二，将 .reloc 节的 VirtualSize 从 0x0122 修改为 0x0126，使 PE 加载器将新增的 4 字节视为该节的有效内容。第三，将可选映像头中的 AddressOfEntryPoint 修改为 0x14122，使程序启动时从寄生代码位置开始执行。

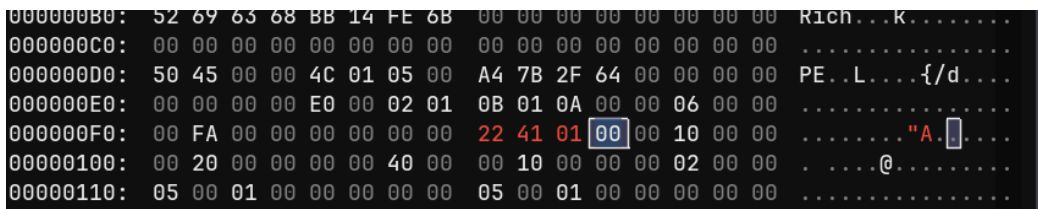
由于 PE 文件中的多字节整数采用小端序存储，三个关键修改分别为：在 0x10122 处写入 EB 02 90 90；在 0x270 处将 22 01 00 00 修改为 26 01 00 00；在 0xF8 处将入口点字段修改为 22 41 01 00。



(a) 写入寄生机器码 EB 02 90 90



(b) 将 .reloc 节 VirtualSize 修改为 0x0126



(c) 将入口点字段修改为 0x14122

图 9 尾区段寄生实验中的三处关键修改

如图 9 所示，本实验先在 .reloc 节空洞起始位置写入 EB 02 90 90，再将 .reloc 节 VirtualSize 扩展到 0x0126，最后将 AddressOfEntryPoint 字段改为 0x14122。三处修改分别对应“写入代码”“扩展有效节大小”和“改变程序入口点”三个环节。由于 Size of Raw Data 仍保持为 0x0200，因此本实验没有增加文件长度，而是在原有节空洞中完成寄生。

1.2.4 寄生结果验证

完成修改后，分别使用 PEView 和 x32dbg 对结果进行验证。PEView 用于静态确认入口点字段是否已经写入 0x14122；x32dbg 用于动态观察程序加载后是否从写入的寄生机器码处开始执行。

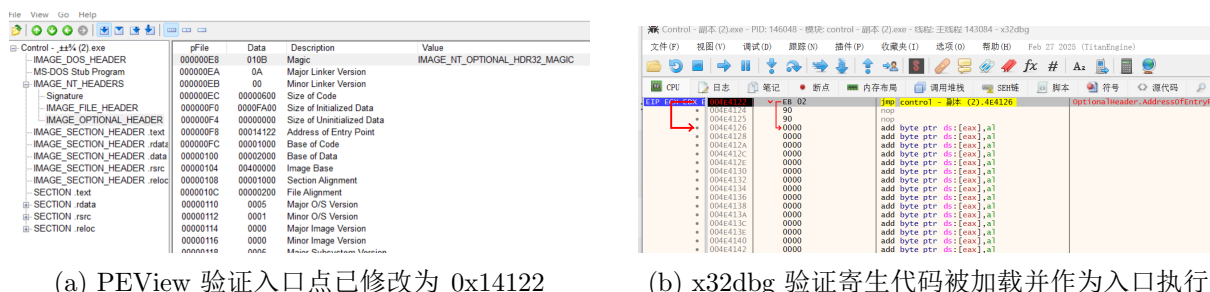


图 10 尾区段寄生代码的静态与动态验证结果

如图 10 所示，左图中 Address of Entry Point 字段已经变为 00014122，说明 PE 文件入口点已经指向 .reloc 节空洞中的寄生代码位置。右图中 x32dbg 停在 004E4122 处，右侧标注当前位置为 OptionalHeader.AddressOfEntryPoint，说明该地址正是程序启动后的入口位置。

结合前文计算可知，寄生代码的 RVA 为 0x14122。调试器中显示的运行时入口地址为 004E4122，说明本次程序实际加载基址为 0x004D0000，因此 $0x004D0000 + 0x14122 = 0x004E4122$ ，与调试器结果一致。

在 004E4122 处可以看到机器码 EB 02，反汇编结果为短跳转指令；其后连续两个字节 90 90 被解释为空操作指令 nop。该结果与实验中写入的 EB 02 90 90 一致，说明寄生机器码已经被 PE 加载器加载到内存，并且程序入口点已经成功转移到该寄生代码处执行。

六、实验结论：

本实验完成了 PE 文件入口点修改和尾区段寄生两个部分，实验结果能够形成较完整的静态与动态验证闭环。

首先，在 PE 文件入口点修改实验中，使用 PEView 定位到原始样本的 AddressOfEntryPoint 字段值为 0x1404。结合 .text 节的 RVA 0x1000 和 Pointer to Raw Data 0x0400，计算得到原入口点对应的文件偏移为 0x0804。随后在副本文件中将入口点字段修改为 0x1419，并通过 PEView 验证 Address of Entry Point 已变为 00001419。最后使用 x32dbg 打开修改后的程序，观察到程序入口处停在 RVA 0x1419，对应机器码为 6A 01，反汇编结果为 push 1。由此可以确认，AddressOfEntryPoint 字段能够直接改变程序装载后的起始执行位置。

其次，在 PE 文件尾区段寄生实验中，使用 PEView 查看最后一个节 .reloc 的节表信息，得到 Virtual Size 为 0x0122，Size of Raw Data 为 0x0200，说明该节存在 0x00DE 字节的空洞空间。根据 Pointer to Raw Data 0x10000 和 .reloc 节 RVA 0x14000，计算出寄生代码的文件偏移为 0x10122，对应 RVA 为 0x14122。随后在 0x10122 处写入 EB 02 90 90，将 .reloc 节 VirtualSize 修改为 0x0126，并将 AddressOfEntryPoint 修改为 0x14122。最终，PEView 验证入口点字段已变为 00014122，x32dbg 验证程序启动后停在 004E4122 处，入口机器码与写入的 EB 02 90 90 一致。由此可以确认，寄生代码不仅被写入文件空洞，而且已经被 PE 加载器加载到内存，并作为新的入口点执行。

综合两个实验可以得出结论：PE 文件的执行流程并不是由文件中代码的物理排列顺序直接决定，而是由 PE 头和节表中的关键字段共同控制。AddressOfEntryPoint 决定程序从哪个 RVA 开始执行；VirtualSize、Size of Raw Data 和 Pointer to Raw Data 则决定节在文件中和内存中的有效范围。若只是简单向文件尾部追加字节，而不修改相关结构字段，新增内容无法保证被加载和执行。只有同时完成“代码写入、节大小描述修正、入口点重定向”三个环节，尾区段寄生才具有可验证的执行效果。

七、总结及心得体会：

这次实验最明显地改变了我对可执行文件的理解。原来我对 exe 文件的认识比较直观，容易把它看成“一段可以运行的代码文件”，似乎只要里面有机器码，程序就能顺序执行。但通过本实验可以看出，PE 文件更像是一份带有严格索引和装载规则的结构化文件。操作系统运行程序时，并不是从文件开头机械读取机器码，而是先解析 PE 头、可选映像头和节表，再按照这些字段把文件内容映射到内存，并从 AddressOfEntryPoint 指向的位置开始执行。

在 3-1 中，入口点从 0x1404 修改到 0x1419 后，程序在 x32dbg 中确实从新的位置开始执行。这一点说明入口点不是一个抽象概念，而是能直接控制程序第一条指令的具体字段。更重要的是，实验中还暴露出 RVA、VA 和文件偏移之间的差异：PEView 中看到的是 RVA，ImHex 中修改的是文件偏移，x32dbg 中看到的是运行时虚拟地址。三个地址如果混在一起，就会出现“明明找到了入口点却改错位置”的问题。这个过程让我意识到，逆向分析首先不是看懂汇编，而是要先搞清楚地址体系。

在 3-2 中，我对“寄生”这个概念的理解也变得更具体。把 EB 02 90 90 写进 .reloc 节空洞只是第一步，如果不修改 VirtualSize，加载器未必会把这些新增字节视为节的有效内容；如果不修改 AddressOfEntryPoint，程序也不会主动跳到这段代码执行。因此，真正关键的不是“把代码塞进去”，而是让 PE 结构承认这段代码，并让执行流到达这段代码。这也解释了为什么指导书中强调直接粘贴代码到文件尾部无效，因为 PE 加载器依据的是结构字段，而不是文件最后多出来了什么内容。

从安全角度看，本实验也说明了 PE 文件结构既是程序正常运行的基础，也是恶意

代码隐藏和劫持执行流可能利用的位置。入口点、节表大小、节权限和映射范围这些字段看起来只是格式信息，但它们实际决定了程序如何被装载、哪些字节会进入内存、执行流会从哪里开始。对防御者来说，分析这些字段可以发现异常入口点、可疑节空洞和不合理的节大小变化；对学习者来说，这比只记忆 PE 结构定义更有意义，因为实验把“字段含义”落实到了真实的地址、字节和调试结果上。

报告评分：

指导教师签字：